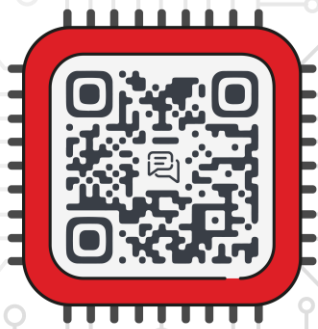




7

7

840 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Thread 2

د. عملي مشترك



برمجة ٣

RB Informatics; 4/12/2025

بداية لتذكر سويا أن هناك طريقتان لإنشاء ال thread

1. إما أن نرث من كلاس thread ونستخدم ميثود ال run()

مثال:

```
// كلاس CookingTask يرث من ال Thread
class CookingTask extends Thread {
    private String task;
    // بانني
    CookingTask(String task) {
        this.task= task;
    }
    // ميثود ال run() لطباعة المهمة وال thread الخاصة بكل مهمة
    @Override
    public void run (){
        System.out.println(task + " is being prepared by " + Thread.currentThread().getName());
    }
}

public class Restaurant{
    public static void main (String[] args) {
        // إنشاء كائن t من ال thread له مهمة معينة
        Thread t1= new CookingTask("Pasta");
        Thread t2= new CookingTask("Salad");
        Thread t3 = new CookingTask("Deserts");
        Thread t4= new CookingTask(" Rice");
        // استخدام ميثود start() للاستدعاء ال thread
        t1.start();
        t2.start();
        t3.start();
        t4.start();
    }
}
```

الخرج:

Pasta is being prepared by Thread-0

Rice is being prepared by Thread-3

Salad is being prepared by Thread-1

Dessert is being prepared by Thread-2

- ملاحظة: تم تنفيذ الـ Threads دون الترتيب الذي كتب به وذلك لأن الـ JVM والـ OS scheduler (مجدول نظام التشغيل) هو الذي يقرر من يشتغل أولاً أو تتم طباعته أولاً.
- أي أن تنفيذ الـ Thread غير مضمون وليس ثابت

2. وإما أن نرث من واجهة Runnable وكما تحدثنا سابقاً هذه الطريقة أفضل عندما نريد أن يرث الكلاس من كلاس آخر

```
class CookingJob implements
Runnable{
    private String task;
    CookingJob( String task){
        this.task= task;
    }
    @Override
    public void run(){
        System.out.println( task+" is being
        prepared by" +
        Thread.currentThread().getName());
    }
}
```

```
public class RestaurantRunnable{
    public static void main (String []args){
        Thread t1 = new Thread(new CookingJob("Soup"));
        Thread t2=new Thread( new CookingJob("Pizza"));
        Thread t3= new Thread( new CookingJob(" Burger"));
        t1.start();
        t2.start();
        t3.start();
    }
}
```

الخرج:

Burger is being prepared by Thread-0

Pizza is prepared by Thread-1

Soup is prepared by Thread-2

Thread priority

كما قلنا فإن جافا تسمح بتشغيل أكثر من Thread في وقت واحد والـ thread scheduler تقرر ترتيب التنفيذ وكل thread لديه أولوية (priority) بين 1 إلى 10 تفضل الـ scheduler الشريد ذات الأولوية المرتفعة وهي التي تنفذ أولاً (ومع ذلك هذا لا يجعل الخرج مضمون 100٪) .

- أخفض أولوية min_priority = 1
- وأعلى أولوية Max_priority = 10 و الأولوية الطبيعية هي 5
- يمكننا استخدام الـ Set والـ Get لإدارة الـ priority.

مثال :

```
class MyThread extends Thread{
    public MyThread( String name){
        super( name);
    }
    public void run(){
        for( int i = 0;i<1000;i++){
            System.out.println( getName()+"
running");
        }
    }
}
```

```
public class TestPriority{
    public static void main( String[] args ){
        MyThread t1 = new MyThread("LOW");
        MyThread t2 = new MyThread("MEDIUM");
        MyThread t3 = new MyThread ("HIGH");
        t1.setPriority(1);
        t2.setPriority(5);
        t3.setPriority(10);
        t1.start();
        t2.start();
        t3.start();
    }
}
```

الخرج:

HIGH running

MEDIUM running

LOW running

⚡ ملاحظة مهمة جدا :

الخرج في هذه الحالة غير مضمون 100% و قد يظهر ترتيب مختلف في كل مرة

⚡ ملاحظة:

لو أكثر من thread حصلوا على نفس ال priority ترتيب التنفيذ سيعتمد على thread scheduler
Synchronization in Java

ال Synchronization (المزامنة) في جافا هي آلية تضمن أن thread واحد فقط يمكنه الوصول إلى شيء مشترك (مثل متغير كائن ميثود)

• لماذا نحتاج المزامنة؟

لأن جافا تدعم multithreading ومن الممكن أكثر من thread أو اثنان يصلان إلى نفس البيانات بنفس الوقت وهذا يسبب :

1. تضارب البيانات Race condition

2. نتائج خاطئة

3. تلف للبيانات

4. تصرفات غريبه بالبرنامج

فالمزامنة تضمن أن thread واحد فقط يستطيع الوصول إلى البيانات المشتركة في لحظة معينة وتعديلها.

لدينا ثلاث طرق لإنشاء المزامنة

1. Synchronized Method :

وهنا نجعل كل الدالة متزامنة او محمية

```
synchronized void print(){
    // الكود الذي نريد حمايته
}
```

2. Synchronized Block :

بدلا من حماية Method كامل نحمي جزءا صغيرا منه .

```
synchronized(this){
    // الجزء المهم فقط
}
```

طبعا بهذه الطريقة نقلل وقت الانتظار ونحسن الأداء .

3. static Synchronization :

إن كانت الدالة static الحماية هنا تصبح على الكلاس كله

```
static synchronized void show(){
}
```

مثال بسيط عن ال Synchronized :

لدينا ثريدين لطباعة الأعداد من 1 ل 5 باستخدام المزامنة بعدم لخبطة الارقام.

```
class Printer {
    public synchronized void
    printNumbers(String name) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(name + " : " +
i);
        }
        System.out.println("-----");
    }
}
```

```
class MyThread extends Thread {
    // انشاء كائن من printer
    Printer p;
    public MyThread(Printer p, String name) {
        this.p = p;
        this.setName(name);
    }
    @Override
    public void run() {
        p.printNumbers(this.getName());
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Printer p = new Printer();
        MyThread t1 = new MyThread(p, "Thread-1");
        MyThread t2 = new MyThread(p, "Thread-2");
        t1.start();
        t2.start();
    }
}
```

الشرح: الثريدين سوف يستخدمان نفس الدالة PrintNumbers() فال Synchronized تجعل ثريد واحد فقط يدخل ويطبّع الأعداد من واحد إلى خمسة وعند الانتهاء يدخل الآخر ويطبّع باقي الأعداد

Thread-1 : 1	Thread-2 : 1
Thread-1 : 2	Thread-2 : 2
Thread-1 : 3	Thread-2 : 3
Thread-1 : 4	Thread-2 : 4
Thread-1 : 5	Thread-2 : 5

إذا خرج سيكون

ولن يحدث الداخل مثل:

Thread-1 : 1

Thread-2 : 1

Thread-1 : 2

Thread-2 : 2



طبعا كما تكلمنا سابقا يمكن استخدام أيضا دالة join() التي تجعل الثريد الحالي ينتظر الثريد الآخر لينتهي من تنفيذ المهمة قبل أن يبدأ هو باستخدامها:

مثال:

```
Thread t1= new Thread()->{
System.out.println("Thread-1"); });
Thread t2 = new Thread ()->{
System.out.println("Thread-2"); });
t1.start();
try {
    t1.join();
} catch (InterruptedException e) {
    e.printStackTrace(); }
t2.start();
```

يعمل Thread-1 أولاً وعند الإنتهاء يعمل ال thread-2

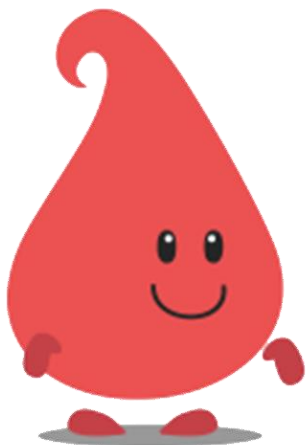
من الممكن أن ترمي دالة join() ال InterruptedException

لذلك يجب أن نتعامل معها فمن الممكن تحديد مدة الإنتظار أو قمنا بمثالنا السابق ب إحاطتها ب (try - catch)

```
t1.join(1000);
```

إضافة أربيسيزية:

ال -> التي تم استخدامها بالكود السابق تعني إشارة اختصار يعني بدلا من كتابة الكود هكذا



```
Thread t1 = new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("Thread-1");
    } });
Thread t2 = new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("Thread-2");
    } });
t1.start();
t1.join();
t2.start();
```

Quiz

Answer the following questions:

1. Which keyword is used to prevent multiple threads from accessing a method simultaneously?

- A) static
- B) final
- C) synchronized
- D) volatile

2. What happens if you call run() instead of start() on a thread object?

- A) It starts the thread normally
- B) It throws an exception
- C) It runs in a new thread
- D) It runs in the current thread

3. Which method is used to pause the execution of the current thread for a given time?

- A) wait()
- B) stop()
- C) yield()
- D) sleep()

4. What is the difference between Runnable and Thread in Java?

- A) Runnable is a class, Thread is an interface
- B) Thread cannot be used with start()
- C) Runnable is implemented, Thread is extended
- D) Thread is always faster

C-D-D-C

الحل:

5. Write a Java program that uses a Runnable implementation to print "Hello from thread" 5 times with a 500ms delay. (Write your code below)

```
class MyTask implements Runnable {  
    @Override  
    public void run() {  
        for (int i= 0; i < 5; i++) {  
            System.out.println("Hello from thread");  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[]  
args) {  
        Thread t = new Thread(new  
MyTask());  
        t.start();  
    }  
}
```

THE END